

Mid-term Assignment — Group Project — PGR207

Deep Learning

Autumn 2026 · Group submission (2–3 students)

Image classification on CIFAR-10: the effect of model architecture, data augmentation, and optimiser on classification performance

1 Overview

You will train deep learning models to **classify natural images into ten categories** using the **CIFAR-10** dataset. You will study **how three factors affect classification performance**:

- **Model architecture**
- **Data augmentation**
- **Optimiser**

Each configuration is trained with **three different random seeds**, and you report your work in a paper following the **IEEE conference format**.

This is a **group assignment**. Each group has **2 or 3 students** and submits **one** paper and **one** set of notebooks. Every member must be able to explain the whole submission, including the parts they did not implement themselves.

For a strong submission, make sure the experiments are **fair and well controlled**. When you study one factor, **keep every other setting identical**. Report the performance variation across the three seeds and take this variation into account when comparing results. Do not claim that one architecture, augmentation strategy, or optimiser is better if the difference is small and falls within the normal seed-to-seed variation.

CIFAR-10 is a widely used dataset with many public implementations and notebooks. Simply reproducing an existing notebook is **not sufficient to pass this assignment**.

2 The dataset: CIFAR-10

A standard benchmark dataset of small natural images in ten mutually exclusive classes, collected and labelled by Krizhevsky [1]. <https://www.cs.toronto.edu/~kriz/cifar.html>

Property	Value
Images	60,000 colour images, 32×32 pixels, 3 channels
Classes	10 classes: airplane, automobile, bird, cat, deer, dog, frog, horse, ship, truck
Split	Official split of 50,000 training and 10,000 test images; balanced , with 6,000 images per class in total
Access	<code>torchvision.datasets.CIFAR10</code> [2] (downloads automatically)
Citation	Krizhevsky [1] — required in your paper

Important

Train **from randomly initialised weights**. **Do not use ImageNet-pretrained weights**. Pre-trained backbones push all architectures close to the same high accuracy on CIFAR-10 and would hide the differences you are asked to measure. If you want to comment on transfer learning, do it as a short side remark in the discussion, not as part of the required experiments.

Keep the input resolution **fixed** across all your experiments. If you resize or upsample the images to fit a particular architecture, apply exactly the same resizing everywhere and state it in the Methods section.

3 The experiments

The task is **multi-class image classification** into the ten CIFAR-10 classes. You will run **three separate factor studies**, all starting from one shared baseline configuration.

3.1 The shared baseline

Define **one baseline configuration** first: **one architecture, one augmentation strategy, one optimiser, and a fixed training budget**. Every factor study then changes **exactly one** of these three factors and leaves everything else untouched. This is called a **one-factor-at-a-time (OFAT)** design, and it is what makes the effect of each factor interpretable.

The following must be **identical in every run** unless it is the factor currently under study:

- **Data splits** and seeds (Section 4).
- Number of **epochs**, or the **early-stopping** criterion, and the **batch size**.
- **Loss function** (cross-entropy is a reasonable default).
- Learning-rate schedule, and how the learning rate was chosen.
- Input resolution and **normalisation statistics**.
- Evaluation code and metric definitions.

Use a **fixed epoch budget** that you can afford to repeat many times. A modest budget applied consistently to every configuration is worth far more than one long, carefully tuned run that cannot be repeated.

3.2 Factor 1: model architecture

Compare **at least three different architectures**. Choose them yourself on the basis of a **literature review**: read work on image classification and on CIFAR-scale training, then select three architectures that are **genuinely different** rather than three sizes of the same family. In your **Related Work** section, explain why you selected them and support the choices with references.

Useful directions to consider: a plain convolutional network as a baseline, a residual network [3], a VGG-style deep stack of convolutions [4], a densely connected network [5], an efficient scaled network [6], or a small vision transformer [7].

Hint: `torchvision.models` [2] provides many classification architectures, and you may instantiate them with random weights (`weights=None`). Note that models designed for 224×224 ImageNet inputs often need a small adjustment for 32×32 inputs — for example a smaller first convolution or a removed max-pooling layer. Whatever you do, **describe the adjustment** and apply the same logic to all architectures.

Also report the **parameter count** of each architecture. A comparison of architectures is much more informative when the reader can see how much capacity each one uses.

3.3 Factor 2: data augmentation

Compare **at least three different augmentation strategies**, applied to the training data only. Your strategies should form a meaningful progression, for example: no augmentation at all, a standard geometric baseline, and a stronger or more modern scheme.

Directions to consider: random crop with padding and horizontal flip as the classical CIFAR-10 recipe, random rotation or small affine shifts, colour jitter, random erasing/Cutout [8], or RandAugment [9]. The survey by Shorten and Khoshgoftaar [10] is a good starting point for the literature. Implementations are available in `torchvision.transforms` [2] and in `Albumentations` [11].

You do **not** need to use label-mixing methods such as mixup or CutMix. Stay with augmentations that transform the **image only** and leave the label unchanged — this keeps the loss computation identical across all your configurations, which is exactly the control the design depends on.

One rule applies throughout: **never augment the validation or test data**. Evaluation must use the same deterministic transform (resize/normalise) in every configuration.

3.4 Factor 3: optimiser

Compare **at least three different optimisers**. Directions to consider: SGD with momentum [12], Adam [13], AdamW [14], RMSprop, Adagrad, or Nesterov-accelerated SGD. The `torch.optim` documentation [15] lists what is available in PyTorch.

Optimisers do not share a sensible learning rate — a rate that works well for SGD is usually far too large for Adam. You therefore have a choice, and you must state which one you made and why:

- **Same learning rate for all optimisers.** Simple and strictly controlled, but it may unfairly handicap one optimiser. If you do this, say so and discuss the consequence.
- **A short learning-rate search per optimiser** (recommended). Try a small grid, for example three candidate rates, and select the best one **using the validation set only**. Report the grid, the selected rate, and the validation numbers you selected on. This is the fairer comparison, and the search itself is a legitimate part of your Methods.

Also report the **weight decay** and any other optimiser hyperparameters, and keep the learning-rate schedule policy the same across optimisers (cosine annealing [16] and step decay are both fine choices).

3.5 How many training runs

Count in two steps: first how many **configurations** the design contains, then how many **training runs** that becomes once each configuration is repeated over seeds.

Step 1 — count the configurations

The baseline is one configuration. Each factor study then reuses the baseline as one of its three levels and adds only the **two remaining** levels as new configurations:

Study	Levels	New configs	What varies
Baseline	1	1	—
1. Architecture	3	2	architecture only
2. Augmentation	3	2	augmentation only
3. Optimiser	3	2	optimiser only
Total		7	configurations

Step 2 — a worked example

The following is one possible instantiation. It is an **illustration of the counting only** — you must choose your own levels from your own literature review.

Suppose a group fixes the **baseline** as: ResNet-18 adapted for 32×32 inputs, augmentation = random crop with 4-pixel padding + horizontal flip, optimiser = SGD with momentum 0.9; trained for 30 epochs at batch size 128 with cosine learning-rate decay. They then choose their three levels per factor (the baseline level is marked $\star$):

- **Architecture:** Simple CNN, ResNet-18 $\star$, DenseNet-121
- **Augmentation:** none, crop + flip $\star$, crop + flip + RandAugment + random erasing
- **Optimiser:** SGD + momentum $\star$, Adam, AdamW

That gives exactly seven configurations:

ID	Architecture	Augmentation	Optimiser	Study
C1	ResNet-18	crop + flip	SGD + mom.	1, 2, 3
C2	Simple CNN	crop + flip	SGD + mom.	1
C3	DenseNet-121	crop + flip	SGD + mom.	1
C4	ResNet-18	none	SGD + mom.	2
C5	ResNet-18	crop + flip + RandAugment + erasing	SGD + mom.	2
C6	ResNet-18	crop + flip	Adam	3
C7	ResNet-18	crop + flip	AdamW	3

C1 is the baseline, and it is the reason the total is 7 and not 9: it serves as a level in all three studies but is trained only once.

Use this table as a **self-check on your own design**: every row must differ from C1 in **exactly one column**. If a row differs in two columns, the comparison against C1 no longer isolates a single factor, and the design is broken.

Step 3 — from configurations to runs

Each of the seven configurations is trained **three times**, once per seed (Section 4):

$$7 \text{ configurations} \times 3 \text{ seeds} = 21 \text{ training runs}$$

So the example above becomes the 21 runs C1-s0, C1-s1, C1-s2, C2-s0, ..., C7-s2. Each run produces one row in your results file: the configuration ID, the seed, and the final metrics.

Step 4 — what this looks like in the paper

Each study becomes one table with three rows, aggregated over the seeds. For Study 1 (architecture):

ID	Architecture	Params	Accuracy (%)	Macro-F1
C2	Simple CNN	0.6 M	78.4 ± 0.5	0.781 ± 0.005
C1	ResNet-18 (baseline)	11.2 M	89.1 ± 0.3	0.890 ± 0.003
C3	DenseNet-121	7.0 M	90.0 ± 0.9	0.899 ± 0.009

The numbers above are fictitious; they illustrate the reporting format, not an expected result. Note what such a table would let you say: the 10.7-point gap between C2 and C1 is far larger than the seed spread and supports a clear claim, whereas the 0.9-point gap between C1 and C3 is the same size as the spread across seeds (± 0.9) and does **not** support a claim that DenseNet-121 is better.

Studies 2 and 3 produce two more tables of the same shape, in which **C1 appears again** with exactly the same numbers. This repetition is intended — it is the fixed reference point that all three studies are measured against, not a duplication error.

If you change the number of levels

The number of configurations for k factors with L_i levels each is $1 + \sum_{i=1}^k (L_i - 1)$. With three factors at three levels this is $1 + 2 + 2 + 2 = 7$. If you added a fourth architecture, it would be $1 + 3 + 2 + 2 = 8$ configurations, and $8 \times 3 = 24$ runs.

More levels or more seeds are welcome if your compute budget allows, but the design must stay controlled. Note that you are **not** asked for the full factorial design of every combination, which would be $3 \times 3 \times 3 = 27$ configurations and 81 runs.

3.6 Optional: interaction study

The OFAT design above measures each factor **at the baseline setting of the other two**. It cannot tell you whether the factors interact — for example, whether strong augmentation helps a small network and a large network by the same amount.

For a stronger submission, add **one full 3×3 grid** over two factors of your choice: 9 configurations, of which several are already trained, so the additional cost is modest. Then answer: is the ranking of the levels of factor A the same at every level of factor B, or do the curves cross?

3.7 Questions to consider

- Which of the three factors has the largest effect on classification performance?
- Is that effect larger than the variation between your three seeds?
- Does augmentation help all architectures, and does it help them equally?
- Does the choice of optimiser change the final accuracy, the speed of convergence, or both?
- Which architecture gives the best accuracy per parameter, or per unit of training time?
- Do the different configurations make the same kinds of mistakes, or do the confusion matrices differ?
- Which CIFAR-10 classes are hardest, and is this consistent across all configurations?
- Does the gap between training and validation accuracy behave as you would expect when augmentation is made stronger?
- If you ran the interaction study: do the factors act independently, or does the best level of one factor depend on the other?

4 Data splits, seeds, and protocol

| Splits

- Use the **official CIFAR-10 split**: 50,000 training and 10,000 test images.
- Hold out a **validation set** from the 50,000 training images — 10% (5,000 images) is a reasonable choice — using a **class-stratified** split.
- Use the validation set for **early stopping, learning-rate scheduling, and any hyperparameter choice**.
- **Never use the test set** for any training or model-selection decision. Evaluate on it once per run, at the end, using the model you selected on validation.
- Create the validation split **once** and **reuse exactly the same split in every configuration**. Do not re-split between experiments.

| Seeds and repeated runs

Every configuration must be trained **three times** with three different random seeds (for example 0, 1, 2). The seed controls weight initialisation, data shuffling, and the random augmentation draws — but **not** the validation split, which stays fixed.

Report every result as **mean $\pm$ standard deviation** over the three seeds. The spread across seeds is your **noise floor**: it tells you how large a difference has to be before it means anything. State the seeds you used, and set them explicitly in the notebook so the runs are reproducible.

5 Metrics: calculating classification performance

CIFAR-10 is a **balanced, ten-class, single-label** problem, so overall accuracy is a meaningful headline number — but it is not enough on its own. You must report metrics that show **per-class** behaviour as well.

Required:

- **Top-1 accuracy** on the test set — the fraction of images classified correctly.
- **Macro-averaged F1** — the unweighted mean of the per-class F1 scores, which gives every class equal weight regardless of how well the model does on the easy ones.
- **Per-class precision and recall** for all ten classes, for at least your best and worst configurations.
- **Confusion matrix** for at least your best and worst configurations, used in the discussion to say **which** classes are confused with which.

Optional, if useful to your argument: balanced accuracy, Cohen’s κ , Matthews correlation coefficient, macro one-vs-rest ROC-AUC, top-2 accuracy, or the mean cross-entropy loss on the test set as a measure of calibration. Justify any metric you add.

| Averaging: state what you did

Micro-, macro-, and weighted averaging give different numbers, and for a balanced test set micro-averaged F1 is identical to accuracy — which is exactly why it adds nothing. State clearly **which averaging you use** for every reported metric, and be consistent across all experiments.

| Aggregating over seeds

Compute the metric **separately for each run**, then average the three resulting values and report the standard deviation. Do **not** pool the predictions of three runs together and compute one metric over the pooled set — that discards exactly the variability you are trying to measure.

For confusion matrices you may either show the matrix of one representative seed (say which) or the element-wise mean over the three seeds (say so).

| Training curves

Log **training and validation loss and accuracy per epoch** for every run. These curves are the main evidence for statements about convergence speed, overfitting, and optimiser behaviour, and they are what makes the optimiser and augmentation studies interesting rather than just a table of final numbers.

| Comparing fairly

- Compare a configuration only against configurations that differ in **one** factor.
- Report the **difference** between configurations next to the seed-to-seed standard deviation, so the reader can judge whether the difference is meaningful.
- If two configurations are within noise of each other, **say so**. Reporting an honest tie is worth more marks than claiming a winner the data does not support.
- Remember that you are making many comparisons. The best of twenty-one runs is expected to look good partly by chance.

6 Group work

- Groups of **2 or 3 students**. One submission per group.
- Divide the work so that everyone trains models, analyses results, and writes. Do not split the group into “the one who codes” and “the one who writes”.
- A natural division is **one factor study per member**, on top of a **shared pipeline** that the group builds together. Building the pipeline jointly is what keeps the experiments comparable.
- Include a **contribution statement** in the paper: for each student ID, one or two sentences on what that person did.
- Every member is responsible for the whole submission and must be able to explain all of it.

7 Deliverables

Submit **one ZIP file** per group, named:

PGR207_2026_MID_<student ID 1>_<student ID 2>[_<student ID 3>].zip

The ZIP file must contain:

- **Paper (PDF)** — Use the IEEE conference template. Maximum **8 pages, including references**.
- **Jupyter notebook(s)** — Include all code and **saved outputs**. The notebooks should run from top to bottom on a clean runtime.
- **Results file** — One CSV or JSON with one row per training run (configuration, seed, and the final metrics), so the tables in the paper can be traced back to the runs that produced them.

The **student IDs of all group members** must appear:

- In the paper instead of your names.
- In the ZIP filename.
- Inside the Jupyter notebook(s).

Do not include the dataset or trained model checkpoints.

8 Structure of the paper

Use the **IEEE conference template** (<https://www.ieee.org/conferences/publishing/templates>). The main focus should be on **Methods, Results, and Discussion & Conclusion**.

- **Title and Authors**
 - Give your paper a suitable title.
 - Use the **student IDs** of all group members as the authors.
- **Abstract**
 - Briefly summarize the problem, methods, main results, and conclusion.
- **Introduction and Related Work**
 - Briefly introduce image classification and the CIFAR-10 benchmark.
 - State your **research questions and hypotheses** — one per factor.
 - Review relevant literature on architectures, data augmentation, and optimisation.
 - Explain and justify your choice of **three architectures, three augmentation strategies, and three optimisers** using the literature.
- **Methods — Main focus**
 - Describe CIFAR-10 and your data preparation and normalisation.
 - Describe the **train/validation/test split** and confirm it is fixed across all experiments.
 - Describe the **baseline configuration** and the **one-factor-at-a-time design**.
 - Describe the architectures, including any modification for 32×32 inputs, and their parameter counts.
 - Describe the augmentation strategies, and confirm they are applied to training data only.
 - Describe the optimisers, their learning rates, and how those rates were chosen.
 - Describe the loss function, epoch budget, batch size, early stopping, and learning-rate schedule.
 - Describe the evaluation metrics and the averaging used.
 - State the **three seeds** and the hardware you trained on.
 - Clearly state what is **fixed** and what **varies** in each of the three studies.
- **Results — Main focus**

- One table per factor study, as **mean $\pm$ standard deviation** over the three seeds.
- **Training curves** for the runs you discuss.
- **Confusion matrices** and **per-class metrics** for at least the best and worst configurations.
- Show how performance changes with **architecture**, with **augmentation**, and with **optimiser**.
- Report **parameter counts** and **training cost** next to accuracy.
- If you ran the interaction study, present the 3×3 grid and state whether the curves cross.
- **Discussion and Conclusion — Main focus**
 - Explain what your results mean.
 - Answer your **research questions** and state whether each **hypothesis** was supported.
 - Rank the three factors by **effect size** and justify the ranking against the seed-to-seed variation.
 - Discuss which differences are meaningful and which are within noise.
 - Discuss the error patterns: which classes are hard, and why they might be confused.
 - Discuss the **limitations** of your study, including: the fixed epoch budget, the fact that each factor was measured at one baseline setting of the others, the limited number of seeds, the small 32×32 resolution, and the many comparisons made against a single test set.
 - Summarize the main conclusions from your experiments.
- **Contribution Statement**
 - For each student ID, one or two sentences on what that person contributed.
- **AI Usage Statement**
 - State whether you used AI tools such as ChatGPT, Copilot, or similar tools.
 - If used, name the tool(s) and briefly explain **how they were used** (e.g., coding, debugging, writing, or language improvement).
 - You are responsible for understanding and verifying any AI-assisted content.
- **References**
 - Use **IEEE reference style**.
 - Include the original **CIFAR-10 technical report** [1].
 - Include the literature used to justify your architectures, augmentations, and optimisers.

9 Resources

Full bibliographic details for everything listed here are given in the References at the end of this document. Cite these sources in your own paper using **IEEE reference style**.

- **Dataset** — CIFAR-10 [1]: cs.toronto.edu/~kriz/cifar.html
- **Architectures and data loading** — TorchVision [2]: pytorch.org/vision/stable/models.html
- **Augmentation** — TorchVision transforms [2] and Albumentations [11]: albumentations.ai
- **Framework and optimisers** — PyTorch [17] and the `torch.optim` documentation [15]: pytorch.org/docs/stable/optim.html
- **Metrics** — scikit-learn [18] and its model-evaluation guide [19]: scikit-learn.org/stable/modules/model_evaluation.html
- **Paper template** — IEEE conference templates [20]: ieee.org/conferences/publishing/templates.html

10 Marking scheme

Marked out of 100: 40% experimental work, 60% manuscript.

Component	Weight	What is assessed
A. Jupyter Notebooks	40%	Experimental work, results, and code
Experimental setup	15%	Correct one-factor-at-a-time design over the three factors (architecture, augmentation, optimiser) from a single shared baseline; fixed data splits reused everywhere; 3 seeds per configuration; everything not under study held constant; no test-set leakage into any training or model-selection decision
Results	15%	All required runs completed (≥ 7 configurations $\times$ 3 seeds) and classification performance correctly calculated : accuracy, macro-F1, per-class metrics, and confusion matrices, reported as mean $\pm$ standard deviation across seeds
Code quality	10%	Clear, readable, and reproducible code that runs from beginning to end; seeding, configuration handling, and the training/evaluation loop are factored so that the three studies share one pipeline
B. Paper	60%	Quality of the IEEE-format manuscript
Paper structure	20%	Clear abstract, research questions and hypotheses, methods, related work (justifying the chosen architectures, augmentations, and optimisers), results, discussion, and conclusion
Results and discussion	25%	Clear tables/figures for all three factor studies; meaningful analysis of the effect size of each factor relative to seed-to-seed variation ; sound discussion of findings, training dynamics, error patterns, and limitations
Completeness	15%	Complete paper, correct IEEE references and formatting, within the 8-page limit , with the contribution statement and AI usage statement included

I Group marking

The group receives **one mark**, awarded to every member, provided the contribution statement shows a reasonable division of the work. If the contribution statement or the submitted material shows that the workload was **clearly unequal**, individual marks may be adjusted. Every member must be able to explain **all** parts of the submission, not only the part they personally implemented.

I Reaching the top band

A submission in the highest band goes beyond running the required configurations. It typically:

- Includes the **optional interaction study** (Section 3.6) and uses it to test whether the factors act independently.
- Argues from **effect sizes and seed variation** rather than from single best numbers, and states plainly when a difference is too small to support a claim.
- Analyses **where** the models fail — confusion between specific CIFAR-10 classes, per-class recall, and example misclassifications — instead of reporting accuracy alone.
- Reports **training cost** (epoch time, parameter count, or total training time) alongside accuracy, and discusses the accuracy–cost trade-off.

Use of AI in preparing this assignment

AI tools were used in preparing this assignment document **only for text clarity and formatting**. They were not used to define the experimental design, the requirements, or the marking criteria.

References

- [1] A. Krizhevsky, “Learning multiple layers of features from tiny images,” University of Toronto, Tech. Rep., 2009. [Online]. Available: <https://www.cs.toronto.edu/~kriz/cifar.html>
- [2] TorchVision maintainers and contributors, *TorchVision: PyTorch’s computer vision library*, 2016. [Online]. Available: <https://github.com/pytorch/vision>
- [3] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2016, pp. 770–778. DOI: 10.1109/CVPR.2016.90
- [4] K. Simonyan and A. Zisserman, “Very deep convolutional networks for large-scale image recognition,” in *International Conference on Learning Representations (ICLR)*, 2015.
- [5] G. Huang, Z. Liu, L. van der Maaten, and K. Q. Weinberger, “Densely connected convolutional networks,” in *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, 2017, pp. 4700–4708. DOI: 10.1109/CVPR.2017.243
- [6] M. Tan and Q. V. Le, “EfficientNet: Rethinking model scaling for convolutional neural networks,” in *International Conference on Machine Learning (ICML)*, 2019, pp. 6105–6114.
- [7] A. Dosovitskiy et al., “An image is worth 16x16 words: Transformers for image recognition at scale,” in *International Conference on Learning Representations (ICLR)*, 2021.
- [8] T. DeVries and G. W. Taylor, “Improved regularization of convolutional neural networks with cutout,” *arXiv preprint arXiv:1708.04552*, 2017.
- [9] E. D. Cubuk, B. Zoph, J. Shlens, and Q. V. Le, “RandAugment: Practical automated data augmentation with a reduced search space,” in *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)*, 2020, pp. 702–703. DOI: 10.1109/CVPRW50498.2020.00359
- [10] C. Shorten and T. M. Khoshgoftaar, “A survey on image data augmentation for deep learning,” *Journal of Big Data*, vol. 6, no. 60, 2019. DOI: 10.1186/s40537-019-0197-0
- [11] A. Buslaev, V. I. Iglovikov, E. Khvedchenya, A. Parinov, M. Druzhinin, and A. A. Kalinin, “Albumentations: Fast and flexible image augmentations,” *Information*, vol. 11, no. 2, p. 125, 2020. DOI: 10.3390/info11020125 [Online]. Available: <https://albumentations.ai/>
- [12] I. Sutskever, J. Martens, G. Dahl, and G. Hinton, “On the importance of initialization and momentum in deep learning,” in *International Conference on Machine Learning (ICML)*, 2013, pp. 1139–1147.
- [13] D. P. Kingma and J. Ba, “Adam: A method for stochastic optimization,” in *International Conference on Learning Representations (ICLR)*, 2015.
- [14] I. Loshchilov and F. Hutter, “Decoupled weight decay regularization,” in *International Conference on Learning Representations (ICLR)*, 2019.
- [15] PyTorch. “`torch.optim` — PyTorch documentation,” Accessed: Aug. 25, 2026. [Online]. Available: <https://pytorch.org/docs/stable/optim.html>
- [16] I. Loshchilov and F. Hutter, “SGDR: Stochastic gradient descent with warm restarts,” in *International Conference on Learning Representations (ICLR)*, 2017.
- [17] A. Paszke et al., “PyTorch: An imperative style, high-performance deep learning library,” in *Advances in Neural Information Processing Systems 32 (NeurIPS 2019)*, 2019, pp. 8024–8035.
- [18] F. Pedregosa et al., “Scikit-learn: Machine learning in Python,” *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011. [Online]. Available: <https://scikit-learn.org/>

- [19] scikit-learn. “Metrics and scoring: Quantifying the quality of predictions,” Accessed: Aug. 25, 2026. [Online]. Available: https://scikit-learn.org/stable/modules/model_evaluation.html
- [20] IEEE. “Conference proceedings templates,” Accessed: Aug. 25, 2026. [Online]. Available: <https://www.ieee.org/conferences/publishing/templates.html>

— *End of the document* —